

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 1
По дисциплине «Обработка изображений в интеллектуальных системах»
Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнил:
Студент 4 курса
Группы ИИ-26
Згера Е. А.
Проверил:
Андренко К.В.

Вариант 5

Цель: научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения

Задачи лабораторной работы

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать `torchvision.datasets`). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (`matplotlib`);
2. Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1;
3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата);
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

5	STL-10 (размеченная часть)	96X96	SGD
---	----------------------------	-------	-----

Код программы

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np
import random
from PIL import Image

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
batch_size = 128
num_epochs = 25
lr = 0.01
num_classes = 10

stl10_classes = [
    "airplane", "bird", "car", "cat", "deer",
    "dog", "horse", "monkey", "ship", "truck"
]

MEAN = [0.4469, 0.4399, 0.4066]
STD = [0.2603, 0.2566, 0.2713]

train_transform = transforms.Compose([
    transforms.RandomCrop(96, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=MEAN, std=STD)
])
```

```

test_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=MEAN, std=STD)
])

train_dataset = torchvision.datasets.STL10(
    root="./data",
    split="train",
    download=True,
    transform=train_transform
)

test_dataset = torchvision.datasets.STL10(
    root="./data",
    split="test",
    download=True,
    transform=test_transform
)

print(f"Train: {len(train_dataset)}, Test: {len(test_dataset)}")

# ВАЖНО ДЛЯ WINDOWS → num_workers=0
train_loader = DataLoader(train_dataset, batch_size=batch_size,
                           shuffle=True, num_workers=0)
test_loader = DataLoader(test_dataset, batch_size=batch_size,
                           shuffle=False, num_workers=0)

class SimpleSTL10CNN(nn.Module):
    def __init__(self):
        super(SimpleSTL10CNN, self).__init__()

        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.pool1 = nn.MaxPool2d(2, 2) # 64 × 48 × 48

        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.conv4 = nn.Conv2d(128, 256, kernel_size=3, padding=1)
        self.pool2 = nn.MaxPool2d(2, 2) # 256 × 24 × 24

        self.conv5 = nn.Conv2d(256, 256, kernel_size=3, padding=1)
        self.pool3 = nn.MaxPool2d(2, 2) # 256 × 12 × 12

        self.fc1 = nn.Linear(256 * 12 * 12, 1024)
        self.fc2 = nn.Linear(1024, num_classes)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = self.pool1(x)

        x = F.relu(self.conv3(x))
        x = F.relu(self.conv4(x))
        x = self.pool2(x)

        x = F.relu(self.conv5(x))
        x = self.pool3(x)

        x = x.view(x.size(0), -1)
        x = self.dropout(F.relu(self.fc1(x)))
        x = self.fc2(x)
        return x

model = SimpleSTL10CNN().to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9)

def evaluate():
    model.eval()
    total = 0
    correct = 0
    loss_sum = 0

    with torch.no_grad():
        for images, labels in test_loader:
            images = images.to(device)

```

```

        labels = labels.to(device)

        outputs = model(images)
        loss = criterion(outputs, labels)
        loss_sum += loss.item() * images.size(0)
        _, predicted = torch.max(outputs, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    return loss_sum / total, 100 * correct / total

train_losses = []
test_losses = []
test_accs = []

for epoch in range(num_epochs):
    model.train()
    running_loss = 0

    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)

    train_loss = running_loss / len(train_dataset)
    test_loss, test_acc = evaluate()
    train_losses.append(train_loss)
    test_losses.append(test_loss)
    test_accs.append(test_acc)

    print(f"Epoch {epoch+1}/{num_epochs} | "
          f"Train Loss: {train_loss:.4f} | "
          f"Test Loss: {test_loss:.4f} | "
          f"Test Acc: {test_acc:.2f}%")

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(train_losses, label="Train Loss")
plt.plot(test_losses, label="Test Loss")
plt.legend()
plt.title("Loss")

plt.subplot(1, 2, 2)
plt.plot(test_accs, label="Test Accuracy")
plt.legend()
plt.title("Accuracy")

plt.show()

def denormalize(img):
    mean = np.array(MEAN)
    std = np.array(STD)
    img = img.numpy().transpose((1, 2, 0))
    img = std * img + mean
    return np.clip(img, 0, 1)

model.eval()
num_samples = 3
sample_indices = random.sample(range(len(test_dataset)), num_samples)

fig, axes = plt.subplots(1, num_samples, figsize=(4 * num_samples, 4))

for ax, idx in zip(axes, sample_indices):
    image, label = test_dataset[idx]

    with torch.no_grad():
        output = model(image.unsqueeze(0).to(device))
        _, pred = torch.max(output, 1)

```

```

true_class = stl10_classes[label]
pred_class = stl10_classes[pred.item()]
correct = pred.item() == label

ax.imshow(denormalize(image))
ax.set_title(f"True: {true_class}\nPred: {pred_class}",
             color='green' if correct else 'red')
ax.axis("off")

plt.tight_layout()
plt.show()

def predict_custom_image(path_or_pil):
    model.eval()

    img = Image.open(path_or_pil).convert("RGB") if isinstance(path_or_pil, str) else
    path_or_pil.convert("RGB")

    preprocess = transforms.Compose([
        transforms.Resize((96, 96)),
        transforms.ToTensor(),
        transforms.Normalize(mean=MEAN, std=STD)
    ])

    input_tensor = preprocess(img).unsqueeze(0).to(device)

    with torch.no_grad():
        probs = F.softmax(model(input_tensor), dim=1).cpu().numpy().flatten()

    pred_label = int(np.argmax(probs))

    fig, axes = plt.subplots(1, 2, figsize=(10, 4), gridspec_kw={'width_ratios': [1, 1.4]})

    axes[0].imshow(img)
    axes[0].axis('off')
    axes[0].set_title(f"Предсказание сети: {stl10_classes[pred_label]}")

    colors = ['red' if i == pred_label else 'steelblue' for i in range(num_classes)]
    axes[1].barh(stl10_classes, probs * 100, color=colors)
    axes[1].set_xlabel("Вероятность, %")
    axes[1].set_xlim(0, 100)
    axes[1].invert_yaxis()

    plt.tight_layout()
    plt.show()

    return pred_label, probs

predict_custom_image(r"1.jpg")

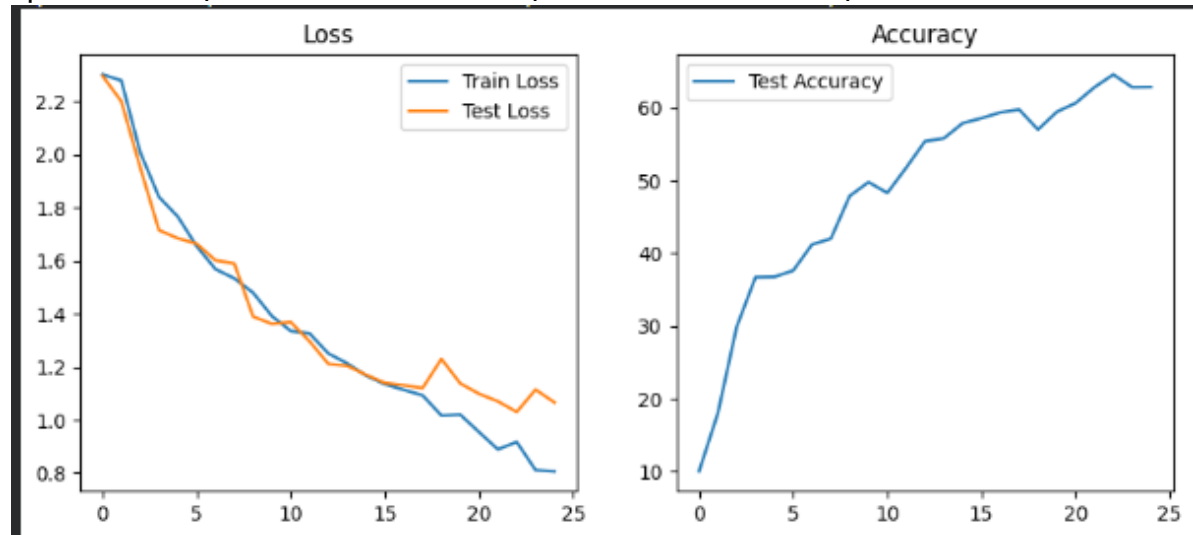
```

Результат работы программы:

Train: 5000, Test: 8000

Epoch 1/25	Train Loss: 2.3009	Test Loss: 2.2960	Test Acc: 10.04%
Epoch 2/25	Train Loss: 2.2786	Test Loss: 2.2001	Test Acc: 18.02%
Epoch 3/25	Train Loss: 2.0092	Test Loss: 1.9525	Test Acc: 29.90%
Epoch 4/25	Train Loss: 1.8399	Test Loss: 1.7148	Test Acc: 36.73%
Epoch 5/25	Train Loss: 1.7659	Test Loss: 1.6843	Test Acc: 36.77%
Epoch 6/25	Train Loss: 1.6547	Test Loss: 1.6639	Test Acc: 37.64%
Epoch 7/25	Train Loss: 1.5681	Test Loss: 1.6014	Test Acc: 41.23%
Epoch 8/25	Train Loss: 1.5334	Test Loss: 1.5893	Test Acc: 42.02%
Epoch 9/25	Train Loss: 1.4792	Test Loss: 1.3894	Test Acc: 47.89%
Epoch 10/25	Train Loss: 1.3911	Test Loss: 1.3620	Test Acc: 49.80%
Epoch 11/25	Train Loss: 1.3350	Test Loss: 1.3686	Test Acc: 48.31%
Epoch 12/25	Train Loss: 1.3260	Test Loss: 1.2960	Test Acc: 51.77%
Epoch 13/25	Train Loss: 1.2503	Test Loss: 1.2109	Test Acc: 55.41%
Epoch 14/25	Train Loss: 1.2130	Test Loss: 1.2041	Test Acc: 55.81%
Epoch 15/25	Train Loss: 1.1672	Test Loss: 1.1701	Test Acc: 57.90%
Epoch 16/25	Train Loss: 1.1347	Test Loss: 1.1394	Test Acc: 58.59%
Epoch 17/25	Train Loss: 1.1136	Test Loss: 1.1310	Test Acc: 59.38%

Epoch 18/25	Train Loss: 1.0929	Test Loss: 1.1201	Test Acc: 59.77%
Epoch 19/25	Train Loss: 1.0169	Test Loss: 1.2299	Test Acc: 57.01%
Epoch 20/25	Train Loss: 1.0201	Test Loss: 1.1378	Test Acc: 59.46%
Epoch 21/25	Train Loss: 0.9543	Test Loss: 1.0986	Test Acc: 60.66%
Epoch 22/25	Train Loss: 0.8895	Test Loss: 1.0702	Test Acc: 62.84%
Epoch 23/25	Train Loss: 0.9172	Test Loss: 1.0304	Test Acc: 64.59%
Epoch 24/25	Train Loss: 0.8113	Test Loss: 1.1145	Test Acc: 62.84%
Epoch 25/25	Train Loss: 0.8065	Test Loss: 1.0662	Test Acc: 62.88%



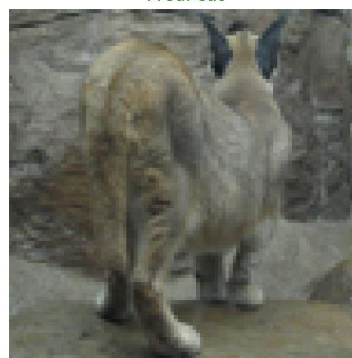
True: deer
Pred: deer



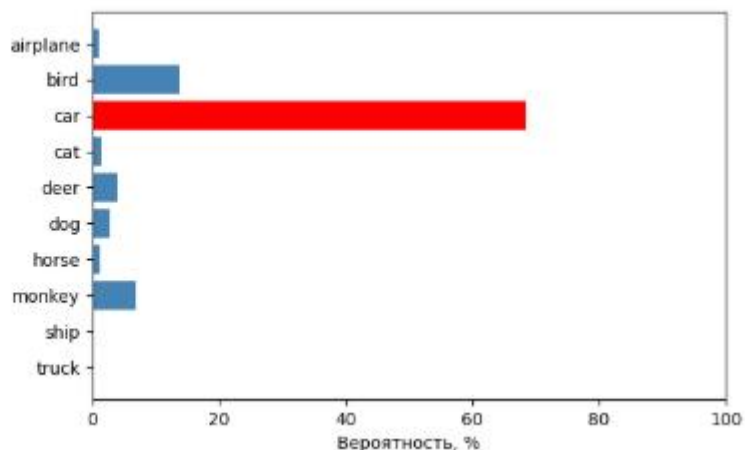
True: ship
Pred: ship



True: cat
Pred: cat



Предсказание сети: car



Свёрточные сети умеют выделять из изображений ключевые элементы — линии, контуры, текстуры — и на их основе различать объекты. Даже минимальная архитектура, построенная только на базовых слоях, уже способна уверенно работать на стандартных датасетах и давать приемлемую точность.

Но такие модели имеют естественные пределы: они часто ошибаются на классах, которые визуально похожи, и хуже справляются с изображениями, отличающимися по стилю от обучающей выборки. Простая CNN хорошо работает в пределах своего датасета, но её способность к обобщению ограничена.

Чтобы приблизиться к уровню современных моделей, нужны архитектурные улучшения: более глубокие сети, стабилизирующие механизмы вроде BatchNorm или остаточных связей, а также большие и разнообразные обучающие наборы. Именно такие элементы позволяют state-of-the-art CNN и трансформерам достигать высокой точности и устойчивости.

Вывод: Проведя данную лабораторную работу я научился конструировать нейросетевые классификаторы и выполнять из обучение на известных выборках компьютерного зрения.